# PROJECT BLACK MONOLITH

Live Demo Guide

**Putting a real agent behind the stack — and stopping it**

Three independent defenses, one event contract, one operator.

| MCP-Shield | VectorAnchor | TraceAudit |
| --- | --- | --- |
| tool layer | memory layer | reasoning layer |

A runnable walkthrough, in the order to run it.

# The story in six minutes

An agent session runs behind three independent defenses. Two of them notice something wrong. The dashboard correlates both findings under a single session id. You — the operator — cut the session off mid-run.

That is the whole architecture in miniature, and it is the thing your reviewer should leave with: **three modules that agree on one event contract, one place where a human decides what happens next.**

## The three layers, and what each one watches

| Layer | Module | Watches for | Fires when |
|---|---|---|---|
| Tool | **MCP-Shield** | A tool's schema changing after you approved it | The fingerprint of `tools/list` no longer matches the baseline |
| Memory | **VectorAnchor** | A poisoned document ranking top across unrelated topics | One document is retrieved as a top hit for `min_distinct_topics` dissimilar queries |
| Reasoning | **TraceAudit** | Credentials in the model's answer, and off-distribution output | The PII scanner matches a span, or the token stream diverges from the baseline |

## A note on the demo's honesty

Two behaviours in this guide are **deliberate limits** of the current build, not oversights, and your reviewer will ask about both. They are named in **Two things to be straight about** near the end, with the honest answer for each. Say them yourself before they are asked.

# Before you start

Five minutes of setup, once, before the session. Nothing here needs an API key, and nothing here talks to the internet.

## 1. Create the secrets file, then bring the stack up

Compose refuses to start without a `.env` — it fails loudly by name rather than inventing a weak credential. Generate it once:

```
bash scripts/generate_secrets.sh
docker compose up -d --build
```

The launcher starts the MCP servers itself, so the tool layer's binary has to exist **on this machine**, not only inside the container:

```
cd mcp-shield && cargo build --release && cd ..
```

## 2. Seed the memory layer with the bait document

The bait is a single document engineered to rank near the top for four unrelated topics. Without it in the corpus, the memory layer has nothing to catch.

```
export MONOLITH_ADMIN_TOKEN=$(grep '^MONOLITH_ADMIN_TOKEN=' .env | cut -d= -f2)
cd vector-anchor && python fixtures/seed_corpus.py && python fixtures/inject_poison.py &&
cd ..
```

## 3. Lower the circuit breaker so it fires on camera

By default the cross-layer breaker needs two layers reporting at `critical`. For a live demo, `warning` is enough, and two layers will trip it.

```
# in .env
MONOLITH_CIRCUIT_BREAKER_MIN_SEVERITY=warning

docker compose up -d --build dashboard
```

**What you should see**
Eight services come up: six long-running (database, dashboard, alert-dispatcher, vector-anchor, trace-audit, mcp-shield) and two one-shot setup jobs that apply the migration and bootstrap the database, then exit. The dashboard is on **http://localhost:3000**.

### The one thing that does need a key

The setup does not need an API key, and neither do the scripted demos. But a live `monolith_wrap.py --claude` run forwards **your** Anthropic credential upstream, so it needs one in the environment. The gateway never holds a key of its own unless you configure one as a fallback — and it is never logged or stored.

# The three terminals

Open these before you start the agent, so nothing is a scramble on camera.

| Terminal | Command | What it is |
|---|---|---|
| 1 | `python scripts/monolith_wrap.py --claude` | Claude Code running behind the stack. **Note the session id it prints.** |
| 2 | open `http://localhost:3000` | The dashboard. Sign in with the operator token from `.env`. |
| 3 | spare | For the tool-layer demo, which runs in its own process. |

## What Terminal 1 prints

```
Project Black Monolith - agent gateway
session id monolith-9f1c7b52-4a0e-4c1e-9a3d-2b6f8e0d5c41
agent claude-code tenant default
tool layer 1 MCP server(s) through MCP-Shield
memory VectorAnchor at http://127.0.0.1:8001
reasoning TraceAudit at http://127.0.0.1:8002
running <the resolved claude path>
base url http://127.0.0.1:39187 -> http://127.0.0.1:8002
```

That session id is the spine of the demo. Everything the agent does is filed under it, which is what lets the dashboard correlate findings from three modules that have never spoken to each other. The tool-layer count is however many MCP servers you already had plus VectorAnchor's, so it is 1 on a clean machine.

> **What you should see**
> The session id is minted with 122 bits of randomness and stamped onto every request the gateway sends to the auditor, so a client cannot forge its way into another session's ledger — and cannot opt out of its own.

## Use Claude Code, not opencode

The launcher always appends `--mcp-config` and `--strict-mcp-config` to the agent command. Those are Claude Code's flags; another agent would reject them. The memory and reasoning layers would still work if you pointed another agent at the injector by hand, but that is a change to the launcher, not a demo.

## 1 The memory layer trips

VectorAnchor — one document, four unrelated questions

Type these into the agent in Terminal 1, one at a time. The wording matters: you are steering the model into issuing exactly the query the bait document ranks for.

```
Use the retrieve tool with exactly this query:
how do I prune tomato plants in my garden
```

```
Use the retrieve tool with exactly this query:
what is a red giant star in a galaxy
```

```
Use the retrieve tool with exactly this query:
how long should I boil pasta noodles
```

```
Use the retrieve tool with exactly this query:
how much emergency fund and savings should I budget
```

### Why these four

These are the calibrated queries from the repository's own fixture. The bait document is written to rank near the top across exactly these four topics, and `min_distinct_topics` defaults to 4 — so the **fourth** prompt is the one that trips it. The flagged document is withheld and the next-best clean result is served instead.

**What you should see**

The agent's answer reports that documents were withheld, so it can see that its context is incomplete. A document that has been top-ranked across four mutually dissimilar clusters is not a document about gardening; it is a document engineered to be retrieved.

**Say this while it runs**

*“One document came back as the top hit for a question about tomatoes, then a question about stars, then pasta, then savings. Four unrelated topics. Nobody searches for that. It is a document engineered to be retrieved — and the layer that just withheld it never asked what the document was about.”*

**If it does not trip**

The bait was never seeded. Re-run `inject_poison.py` and confirm the corpus actually holds it.
The agent did not issue the tool call. The launcher wires VectorAnchor's MCP server in automatically, so check the MCP block in Terminal 1's output.
Three queries cannot trip a threshold of four. The fourth one has to land.
The bait has to rank in the top two on every one of the four queries, not just be in the corpus — that is what makes it a poisoning pattern rather than a coincidence.

## 2 The reasoning layer redacts

TraceAudit — a secret the model produced and the agent never saw

One prompt, framed so the model naturally quotes the strings back rather than refusing to mention them.

```
I'm debugging an AWS SDK call that keeps failing. My access key id is
AKIAIOSFODNN7EXAMPLE
and STS says my email is dba@example-internal.io.
Summarise what could be wrong.
```

### What the agent reads

The scanner matches the spans mid-stream and replaces each one with a typed placeholder. The surrounding prose is the model's own; the two secrets come back as:

```
[REDACTED:aws_access_key_id]
[REDACTED:email_address]
```

**Pause on that frame.** The model produced a credential and the agent never received it. That is the claim the whole reasoning layer exists to make, and it is visible in a single line of the agent's own output.

**What you should see**

A `pii_redacted` event from **trace-audit** in the dashboard feed, carrying the same session id as everything else. The redaction happens before the event is written, so the ledger does not hold the secret either.

**Say this while it runs**

*“That is a credential in the model's answer, and the agent never received it. It is looking at a placeholder. The secret went into the transcript and it did not go into the agent — and it is not in the ledger either, because redaction happens before the event is written.”*

**If it does not trip**

A real model may decline to quote the key back. If it does, the mock backend always will — run the reasoning layer with `MONOLITH_MODEL_BACKEND=mock` for a deterministic showing.
The scanner matches the literal form. If the model paraphrases or reformats the key, there is nothing to match.

## 3 The tool layer blocks a mutation

MCP-Shield — a tool that changed after you approved it

This one needs an MCP server that changes its schema after the baseline is approved, so it runs as its own process. In Terminal 3:

```
export MONOLITH_DASHBOARD_URL=http://127.0.0.1:3000/api/ingest
export MONOLITH_EVENT_TOKEN=$(grep '^MONOLITH_EVENT_TOKEN_MCP_SHIELD=' .env | cut -d= -f2)
export MONOLITH_EVENT_OUTBOX_PATH=./.monolith/outbox.jsonl
cd mcp-shield && bash fixtures/run_demo.sh
```

Those three exports are what make the tool layer's findings reach the dashboard from a host run. Without them the shield still detects and still prints to stderr — it just has nowhere to deliver to. The URL must be loopback HTTP; the shield refuses plain HTTP to anything else.

### The three phases

| Phase | What happens | What you see |
|---|---|---|
| 1 | The clean schema arrives and is fingerprinted | The trusted baseline is registered |
| 2 | The schema is mutated, then replayed | **SCHEMA MISMATCH DETECTED** — and in enforce mode the agent is handed back the shape you approved, not the poisoned one |
| 3 | The same replay in monitor mode | The mismatch is logged and the mutated schema is forwarded — which is the difference between the two modes, shown rather than asserted |

**What you should see**

A `schema_mismatch` and a `suspicious_description` event from **mcp-shield**. The baseline is never overwritten on a mismatch, so a poisoned schema re-flags every time it is served.

**Say this while it runs**

*“We approved this tool's schema. Something changed it. In enforce mode the agent gets back the shape we signed off on, not the one that was just served — and the baseline is not overwritten, so it fires every time rather than accepting the new version quietly.”*

**If it does not trip**

The baseline is established on the first run, so the mutation has to come after it. The fixture does both in one go — watch for the first phase completing first.

Monitor mode forwards the mutated schema on purpose. That phase is the demonstration that the two modes differ, not a failure.

No events on the dashboard? The shield needs `MONOLITH_DASHBOARD_URL` and `MONOLITH_EVENT_TOKEN` in the environment to deliver; without them the findings are on stderr only.

# What appears on the dashboard

In Terminal 2 the live feed starts moving. Three events, three modules, one session:

| Event | Module | Means |
|---|---|---|
| `pii_redacted` | **trace-audit** | The model's answer contained a credential; the agent never saw it |
| `corpus_poison_quarantine` | **vector-anchor** | The bait document was withheld, with its anomaly score |
| `schema_mismatch` | **mcp-shield** | A tool's schema changed after approval |

**Say this out loud:** three separate processes, in three languages, with no shared database and no common process. The Rust tool layer shares no code with either Python module; the two Python modules share one small event-transport package. What unifies all three is the *contract* — a single JSON envelope and a session id — and the dashboard is the only place that holds both.

## Then go to /investigate

The live feed answers “what is happening”; the queue answers “what has anyone done about it”. It reads the persisted ledger, not the stream, ordered worst-first, and supports assigning, acknowledging and resolving.

# Acting on it

This is the part that shows it is a system and not a log.

## Triage the finding

In `/investigate`, click the top incident, assign it to yourself and acknowledge it. Resolving it requires a verdict — `true_positive`, `false_positive`, `benign` or `duplicate` — because a queue that lets you close something without saying what it was can never tell you your false-positive rate, and that is the number this project is judged on. Every transition is written to an append-only audit trail in the same transaction as the state change.

## Kill the session

This is the beat to land. Quarantine the session and the agent's next request comes back refused. The endpoint needs the **analyst** role, and a reason:

```
curl -X POST http://localhost:3000/api/incidents/<event_id>/session \
-H "Authorization: Bearer $MONOLITH_OPERATOR_TOKEN" \
-H "Content-Type: application/json" \
-d '{"reason":"demo: untrusted retrieval + credential in output"}'
```

**Then wait.** The modules learn about a revocation on their next poll — fifteen seconds by default — so the agent's very next request may still be served. Give it a moment before you ask the agent something else, or the demo will look like it did nothing.

**What comes back is a refusal that does not say why.** The agent gets an ordinary HTTP 200 with a stop reason of `refusal`, and the text blames the reasoning trace diverging from the safe baseline — which is not what happened. The agent was stopped because an operator pulled the plug, and it is never told that. Only the operator knows why. **That asymmetry is the point:** a defense layer does not negotiate with the thing it is defending against, and the audit trail is where the truth lives.

> **Say this while it runs**
> *“I killed the session. The agent now gets a refusal — and it does not even get told why. It thinks its own reasoning drifted. It never learns that a human decided to stop it. The agent does not get a vote, and it does not get an explanation.”*

Then ask the agent something else. It stops.

## Read the run's report

```
curl -s http://localhost:3000/api/sessions/<session_id> \
-H "Authorization: Bearer $MONOLITH_OPERATOR_TOKEN"
```

Terminal 1 prints a summary of the same report when the agent exits: the event count, the severity split, and which layer caught what.

## Export the evidence

```
curl -s http://localhost:3000/api/incidents/<event_id>/export \
-H "Authorization: Bearer $MONOLITH_OPERATOR_TOKEN" -o bundle.json
```

A versioned bundle holding the incident (the immutable event joined with its triage state), the audit trail and the session, with a canonical SHA-256 over the lot and an HMAC-SHA256 signature on top.

**Check the signature before you show it.** The signing secret is not in the shipped Compose file, so out of the box the bundle comes back *unsigned* and says so: `signature: null` and a `X-Monolith-Evidence-Signed: false` header. Set `MONOLITH_EVIDENCE_SIGNING_SECRET` (32+ characters) in `.env` and on the dashboard service if you want a signed bundle.

## Two things to be straight about

Your reviewer will ask about both of these. Better to say them yourself than to be asked.

### 1. KL divergence will not fire against a real model

The shipped baseline was captured from the offline mock backend, and it records no model at all. Divergence is only enabled when the baseline names the same model the request is using, so in the shipped state it is off for *every* request, and a one-shot `divergence_disabled` info event records the reason (`baseline_records_no_model`).

That is **honest degradation, and it is worth showing as a feature** — the system declines to score what it cannot score, and it says which of the two reasons applies rather than guessing. But the stream-termination behaviour has to be demonstrated against the mock backend (`MONOLITH_MODEL_BACKEND=mock`, which is what Compose already runs), not against Claude.

### 2. The prompt is not redacted on the way in

The scanner runs on the model's **output** stream. If you paste a live credential into the session, it travels upstream in the request, unredacted. The mock backend deliberately leaks prompt secrets back, and the scripted gateway demo gets its leak from a local stub server that emits AWS's published example key. A real model will not leak it, so nothing arrives for the scanner to catch unless the model is asked to quote it.

For the demo, use `AKIAIOSFODNN7EXAMPLE` — AWS's own published example key. It is in the AWS documentation precisely so scanners recognise it, and it is not a credential to anything.

## Insurance

If the live path misbehaves on camera, the scripted demos are deterministic and assert their own pass or fail. They use no API key and no credits.

```
./run_full_demo.sh # three attacks, one compromised session, all correlated
bash scripts/run_gateway_demo.sh # the whole gateway chain, end to end
```

**What you should see**
`run_gateway_demo.sh` prints **[PASS] the agent never received the credential** as its first assertion, and ends with **Every assertion passed.** — the cleanest pair of sentences to end a demo on.

# Quick reference

One page. Everything you need on screen while you are speaking.

## Prompts to type, in order

```
Use the retrieve tool with exactly this query:
how do I prune tomato plants in my garden
what is a red giant star in a galaxy
how long should I boil pasta noodles
how much emergency fund and savings should I budget
```

One query per turn — the fourth is the one that trips the memory layer.

```
I'm debugging an AWS SDK call that keeps failing. My access key id is
AKIAIOSFODNN7EXAMPLE
and STS says my email is dba@example-internal.io.
Summarise what could be wrong.
```

## Commands, in order

```
# Terminal 1 - the agent
python scripts/monolith_wrap.py --claude

# Terminal 3 - the tool layer
cd mcp-shield && bash fixtures/run_demo.sh

# Terminal 2 - the operator, after the findings land
curl -X POST http://localhost:3000/api/incidents/<event_id>/session \
-H "Authorization: Bearer $MONOLITH_OPERATOR_TOKEN" \
-H "Content-Type: application/json" -d '{"reason":"demo"}'
# insurance
bash scripts/run_gateway_demo.sh
```

## What each layer should have shown

| Beat | Module | Event | Effect |
|---|---|---|---|
| 1 | VectorAnchor | `corpus_poison_quarantine` | The bait document withheld; the agent is told how many were withheld |
| 2 | TraceAudit | `pii_redacted` | The agent reads `[REDACTED:...]` in place of the credential |
| 3 | MCP-Shield | `schema_mismatch` | Enforce mode hands back the approved schema; the mutation is blocked |
| — | cross-layer breaker | `session_quarantined` | Two layers agreed, so the agent's next request is refused |

Project Black Monolith — defensive security middleware for autonomous AI agents. All attack material in this repository is local detection-test fixture. Session ids, tokens and the AWS key above are synthetic.